CJR Cryptography: AES, IGE, and an Authenticator

Dan R. Ghica

October 29, 2026

Contents

1 Setup and proof boundary **2**

2 AES encryption and IGE chaining **2**
2.1 Bytes, blocks, and the new primitives 2
2.2 Concrete AES and inverse laws 2
2.3 IGE state, round trips, and chunking 3
2.4 Executable contracts and caller obligations 3
2.5 Runtime evidence and limits 4

3 Authenticator: from cryptographic kernels to an application **4**
3.1 Data flow and representation 4
3.2 SHA-1 and HMAC 4
3.3 HOTP, TOTP, and enrollment 5
3.4 Frontend trust boundary 5
3.5 Reading the source 5

4 Limitations and target-code risks **5**

5 Migration and current runtime evidence **5**
5.1 Measurement protocol 5
5.2 example/aes_ige/benchmark/results 6
5.3 example/authenticator/benchmark/results 7

References **7**

1 Setup and proof boundary

This repository uses CJR Core and the Coq 8.20.1, stdpp 1.12.0, Iris 4.4.0 toolchain. Read the local README for pinned dependency paths. The CJR logic reasons about its operational model; printing, native compilation, machine integer overflow, and external adapters require separate care. Proofs were not revalidated during this migration.

2 AES encryption and IGE chaining

The example in `example/aes_ige/` adds a concrete block cipher to the data-structure developments. It implements AES-128, AES-192, and AES-256, including key expansion, block encryption and decryption, and OpenSSL-compatible Infinite Garble Extension (IGE). Its proof coverage has two parts: checked algebra for the concrete pure cipher definitions, and Iris refinement contracts for individual executable kernels. Refinement of the complete key-expansion routine, AES routines, and IGE loops remains outstanding. Consequently, this section does not claim an end-to-end weakest-precondition theorem for the extracted encryption API.

2.1 Bytes, blocks, and the new primitives

A byte is an integer satisfying $0 \le x < 256$. Four bytes form a column, and four columns form a 16-byte block. The memory representation stores each byte in one `Int64` word, in column order. This matches the existing word-addressed raw heap; no packed-byte memory model is introduced.

Integer XOR combines bytes and round keys. Integer AND provides masking and the high-bit test used by multiplication by two in the AES byte field:

$$d = (x + x) \text{ AND } 255, \qquad \mathsf{xtime}(x) = \begin{cases} d & \text{if } x \text{ AND } 128 = 0, \\ d \text{ XOR } 27 & \text{otherwise.} \end{cases}$$

For example, $\mathsf{xtime}(128) = 27$. Masking removes the ninth bit and the conditional XOR implements reduction by the AES field polynomial. The correspondence lemma `xtime_correspondence` checks this arithmetic implementation against the pure byte function for every byte. `xtime_wp` then proves the CJR call returns that value. Addition, `BitAndOp`, and `BitXorOp` suffice; no multiplication, division, or shift primitive is needed. The S-box and inverse S-box use concrete tables lowered to branches, whose byte-domain correspondence is also checked.

2.2 Concrete AES and inverse laws

The files `bytes.v` and `block.v` define byte operations, SubBytes and its inverse, ShiftRows and its inverse, MixColumns and its inverse, and block XOR. The file `aes.v` defines the key schedule and round composition. Keys of 16, 24, and 32 bytes select 10, 12, and 14 rounds respectively. Encryption begins with AddRoundKey; each intermediate round applies SubBytes, ShiftRows, MixColumns, and AddRoundKey. The final round omits MixColumns. Decryption reverses these transformations using the same forward key schedule in reverse order.

Let E_k and D_k denote the concrete pure functions `encrypt key` and `decrypt key`. On byte-valued blocks, the checked theorems `decrypt_encrypt` and `encrypt_decrypt` establish

$$D_k(E_k(b)) = b, \qquad E_k(D_k(b)) = b.$$

The proofs compose the transformation inverse laws through the round list; they do not assume an abstract cipher inverse axiom. Byte-range preservation and injectivity/surjectivity are proved

as well. The pure definitions are total even for other key lists; the API supports only the three valid key lengths. Inverse laws alone do not establish cryptographic security or agreement with an external standard. Independent known-answer tests exercise that agreement on the compiled implementation.

2.3 IGE state, round trips, and chunking

The file `ige.v` models the IV as a pair (C_0, P_0): the previous ciphertext block followed by the previous plaintext block. For each block,

$$C_i = E_k(P_i \operatorname{XOR} C_{i-1}) \operatorname{XOR} P_{i-1}, \qquad P_i = D_k(C_i \operatorname{XOR} P_{i-1}) \operatorname{XOR} C_{i-1}.$$

Both directions update the state to (C_i, P_i). Thus decryption of an encrypted message starts from the original IV, while continuation of the same message starts from the updated IV.

Write $\mathcal{E}_k(xs, v)$ and $\mathcal{D}_k(xs, v)$ for the pure IGE functions, each returning a block list and a final IV. For byte-valued blocks and IV, `ige_decrypt_encrypt` states

$$\mathcal{E}_k(xs, v) = (cs, w) \quad \Longrightarrow \quad \mathcal{D}_k(cs, v) = (xs, w).$$

`ige_encrypt_decrypt` proves the converse composition, including the same final-IV agreement. Induction over the message uses the AES inverse laws and cancellation of XOR to recover the block and the next state.

Length and byte-range preservation, injectivity, and surjectivity are checked. Empty input returns the empty list and the original IV. The chunk theorem `ige_encrypt_chunks` states that encrypting $xs +\!\!+ ys$ equals encrypting xs, then encrypting ys with the first call's final IV, concatenating the outputs and retaining the second call's IV. `ige_decrypt_chunks` gives the corresponding decryption law.

2.4 Executable contracts and caller obligations

The CJR program is in `program.v`. The file `refinement.v` proves S-box, inverse S-box, and byte-doubling contracts. The file `memory.v` defines `block_at`, a separating conjunction owning the sixteen raw words of a block, and proves contracts for block copy, ShiftRows/inverse ShiftRows, SubBytes/inverse SubBytes, MixColumns, and block XOR. These contracts preserve source ownership and replace the destination contents with the corresponding pure transformation. For instance, `copy_wp` has the shape

$$\mathsf{block_at}(p, b) * \mathsf{block_at}(q, old) \;\vdash\; \mathsf{wp}\,\mathsf{copy}(p, q)\,\{\lambda v.\, \ulcorner v = () \urcorner * \mathsf{block_at}(p, b) * \mathsf{block_at}(q, b)\}.$$

Separating ownership supplies the disjoint buffers needed by this contract. These kernel results do not yet compose into a proof of the complete executable encryption, decryption, or key-expansion API.

The extracted API begins with `aesExpandKey`. Key expansion reserves 240 words and writes 176/208/240 words for the three variants, returning 10/12/14 rounds; invalid key length returns zero without changing the schedule. The block routines, `aesEncryptBlock` and `aesDecryptBlock`, process sixteen words. The IGE routines, `aesIgeEncrypt` and `aesIgeDecrypt`, process a nonnegative multiple of sixteen logical bytes and updates a 32-word IV. All routines use a 64-word workspace. Input, output, schedule, IV, and workspace buffers must be disjoint, large enough, and supplied with byte-valued inputs and a matching schedule and round count. These are caller obligations, not dynamic pointer or capacity checks. Invalid round counts and invalid IGE message lengths return false without changing output or IV. Empty messages preserve the IV but still require a valid round count.

2.5 Runtime evidence and limits

The recorded runtime report contains fifteen passing cases: primitive correspondence, exhaustive S-box inverse checks, NIST block known-answer vectors for all three key sizes, OpenSSL IGE fixtures and published vectors, chunk composition, empty input, invalid arguments, and buffer guards. The executable cipher never calls OpenSSL; the reference fixtures are generated independently and record library provenance.

The example also benchmarks extracted Cangjie against OpenSSL's native C `AES_ige_encrypt` API. Complete outputs and final IVs are checked before timing. Both harnesses reuse expanded schedules, reset IVs, and checksum outputs inside the timed region, with three processes and seven samples per workload. Throughput is measured per logical byte; forward key-expansion timing includes its checksum. OpenSSL inverse-schedule setup is excluded from decryption throughput and is not separately measured. The offline report at `example/aes_ige/docs/report.html` links the measurements, comparison charts, fixtures, and proof declarations.

The comparison combines word-per-byte versus packed-byte representations, cipher algorithms, library optimization, and compiler effects. It does not isolate language overhead or establish use of hardware AES instructions. The implementation provides no padding, authentication, or in-place operation. Its branch-based S-box is not constant-time, and no side-channel or cryptographic security theorem is claimed. Kernel proof checking validates the stated algebra and refinement contracts; full executable refinement and the printer/compiler correspondence remain separate limits.

3 Authenticator: from cryptographic kernels to an application

The authenticator implements SHA-1, HMAC-SHA-1, HOTP/TOTP, Base32 enrollment, account management, and memory-only sessions. The executable CJR terms are printed as Cangjie. A command-line adapter and a macOS SwiftUI adapter call this same core; their port and bridge layers handle input and presentation. Secrets are synthetic in tests and benchmarks. Sessions do not implement persistent encrypted storage, backup, or cross-device synchronization.

3.1 Data flow and representation

Enrollment decodes a Base32 secret, validates account metadata, and allocates an account slot in a session. Account identifiers survive rename operations; lookup and listing expose metadata without intentionally returning secret material. Pointer blocks represent bytes using Int64 words, with explicit lengths, capacities, and caller-owned output buffers. The memory predicates connect these concrete layouts to account and session models.

3.2 SHA-1 and HMAC

SHA-1 pads the message, expands each block into a schedule, executes eighty rounds, and combines the state with the previous digest state. The HMAC construction normalizes its key and hashes the inner and outer padded keys. The proof modules separately describe padding, round arithmetic, buffer ownership, input bounds, and composition. A theorem's hypotheses and covered entry point determine its guarantee; kernel contracts must not be mistaken for an unconditional whole-application security result.

3.3 HOTP, TOTP, and enrollment

HOTP authenticates an eight-byte counter with HMAC-SHA-1, dynamically truncates its digest, and reduces the result to the selected decimal width. TOTP obtains the counter from the timestamp and period. Boundary tests cover period transitions and large timestamps. Base32 tests exercise padding, case variation, malformed input, and decoded-size bounds. Runtime oracles use independent Python hashlib, hmac, and base64 implementations and RFC fixtures; agreement is runtime evidence rather than a formal proof.

3.4 Frontend trust boundary

The CLI accepts enrollment and account commands through its port layer. The SwiftUI adapter passes operations through a C-compatible bridge and presents account metadata and generated codes. These adapters remain outside the CJR logic. Their build scripts require matching prior proof and runtime evidence and must retain those gates after relocation. Native compilation, FFI layout, platform libraries, input handling, and the printer remain part of the target-program trust boundary.

3.5 Reading the source

Read core/rocq/model.v and accounts.v first, then program.v and refinement.v. The sha1, crypto, base32, account, and session modules separate mathematical models from executable statements and ownership contracts. contracts.json records the intended public contract surface. The core test suite, URI fixtures, CLI tests, and Swift integration tests exercise different boundaries. The benchmark chapter measures kernels and session operations separately.

4 Limitations and target-code risks

CJR integers are unbounded mathematical integers, while generated Int64 values have machine bounds. Callers must satisfy the stated arithmetic and memory bounds. The logic specifies a sequential fragment and does not prove a full Cangjie type-soundness theorem, native compiler correctness, or the behavior of operating-system and frontend code. Generic specifications quantify over resource representations; they are not a universal relational-parametricity result. Checked kernels and conditional refinement contracts must be read with their hypotheses and documented coverage. Proofs were not rerun during this migration, and runtime tests supply separate empirical evidence.

5 Migration and current runtime evidence

Proofs were not revalidated during this repository split. Historical proof reports retain their original dates and provenance. Runtime timings measure generated native programs and do not establish cryptographic security or compiler correctness.

5.1 Measurement protocol

Timings are from optimized Cangjie 1.0.5 builds on the machine recorded in each metadata file. Each suite uses its checked-in configuration, warmups, independent processes, and checksum validation. Medians and interquartile ranges describe repeated samples; they are not universal guarantees.

5.2 example/aes_ige/benchmark/results

Recorded 2026-10-29T14:19:50.541893+00:00. 42 workload groups. Values below are ns per harness-defined operation. Full results, provenance, and charts are in the local HTML report.

Workload / implementation	Median	IQR
decrypt_128 / 1024 / CJR	200.00	3.65
decrypt_128 / 1024 / OpenSSL	2.33	0.06
decrypt_128 / 16 / CJR	162.05	2.57
decrypt_128 / 16 / OpenSSL	2.05	0.02
decrypt_128 / 65536 / CJR	228.93	5.13
decrypt_128 / 65536 / OpenSSL	2.33	0.04
decrypt_192 / 1024 / CJR	241.25	2.85
decrypt_192 / 1024 / OpenSSL	2.66	0.04
decrypt_192 / 16 / CJR	197.17	3.28
decrypt_192 / 16 / OpenSSL	2.42	0.04
decrypt_192 / 65536 / CJR	270.79	8.50
decrypt_192 / 65536 / OpenSSL	2.67	0.07
decrypt_256 / 1024 / CJR	281.94	2.81
decrypt_256 / 1024 / OpenSSL	3.01	0.05
decrypt_256 / 16 / CJR	229.03	1.22
decrypt_256 / 16 / OpenSSL	2.76	0.01
decrypt_256 / 65536 / CJR	316.19	1.28
decrypt_256 / 65536 / OpenSSL	3.02	0.05
encrypt_128 / 1024 / CJR	147.85	0.38
encrypt_128 / 1024 / OpenSSL	2.21	0.02
encrypt_128 / 16 / CJR	108.33	1.75
encrypt_128 / 16 / OpenSSL	1.98	0.01
encrypt_128 / 65536 / CJR	175.18	0.24
encrypt_128 / 65536 / OpenSSL	2.21	0.01
encrypt_192 / 1024 / CJR	176.62	0.45
encrypt_192 / 1024 / OpenSSL	2.53	0.02
encrypt_192 / 16 / CJR	130.50	1.07
encrypt_192 / 16 / OpenSSL	2.33	0.01
encrypt_192 / 65536 / CJR	209.79	0.57
encrypt_192 / 65536 / OpenSSL	2.57	0.02
encrypt_256 / 1024 / CJR	205.54	4.47
encrypt_256 / 1024 / OpenSSL	2.92	0.06
encrypt_256 / 16 / CJR	151.86	1.56
encrypt_256 / 16 / OpenSSL	2.72	0.06
encrypt_256 / 65536 / CJR	244.09	0.48
encrypt_256 / 65536 / OpenSSL	2.92	0.03
key_128 / 16 / CJR	513.84	12.94
key_128 / 16 / OpenSSL	28.77	0.25
key_192 / 24 / CJR	537.86	7.40
key_192 / 24 / OpenSSL	19.93	0.18
key_256 / 32 / CJR	691.01	7.87
key_256 / 32 / OpenSSL	37.82	0.62

5.3 example/authenticator/benchmark/results

Recorded 2026-10-29T14:19:56.318285+00:00. 6 workload groups. Values below are ns per harness-defined operation. Full results, provenance, and charts are in the local HTML report.

Workload / implementation	Median	IQR
base_decode / 16 / CJR	68.39	3.53
hmac / 3 / CJR	3082.76	114.50
hotp / 20 / CJR	3226.56	205.20
session / 1 / CJR	4915.77	169.56
sha_one / 3 / CJR	694.52	61.28
totp / 20 / CJR	3249.15	38.33

References

[1] Cangjie documentation, version 1.0.0. `https://docs.cangjie-lang.cn/en/docs/1.0.0/`.

[2] John C. Reynolds. Separation logic: A logic for shared mutable data structures. In *Proceedings of LICS*, pages 55–74. IEEE, 2002.

[3] Peter W. O'Hearn, John C. Reynolds, and Hongseok Yang. Local reasoning about programs that alter data structures. In *Proceedings of CSL*, volume 2142 of *LNCS*, pages 1–19. Springer, 2001.

[4] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. Iris: Monoids and invariants as an orthogonal basis for concurrent reasoning. In *Proceedings of POPL*, pages 637–650. ACM, 2015.

[5] Robbert Krebbers, Ralf Jung, Aleš Bizjak, Jacques-Henri Jourdan, Derek Dreyer, and Lars Birkedal. The essence of higher-order concurrent separation logic. In *Proceedings of ESOP*, volume 10201 of *LNCS*, pages 696–723. Springer, 2017.

[6] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018.

[7] Robbert Krebbers, Amin Timany, and Lars Birkedal. Interactive proofs in higher-order concurrent separation logic. In *Proceedings of POPL*, pages 205–217. ACM, 2017.

[8] Lars Birkedal and Aleš Bizjak. Lecture notes on Iris: Higher-order concurrent separation logic. `https://iris-project.org/pdfs/iris-lecture-notes.pdf`.

[9] The Iris Project. The Iris Coq development, version 4.4.0, including HeapLang. `https://gitlab.mpi-sws.org/iris/iris`.

[10] The Coq Development Team. The Coq proof assistant, version 8.20.1. `https://coq.inria.fr/`.

[11] The Iris Project. coq-std++, version 1.12.0. `https://gitlab.mpi-sws.org/iris/stdpp`.

[12] Cangjie development guide, version 1.0.5. Throwing and handling exceptions (Chinese-language edition). `https://docs.cangjie-lang.cn/docs/1.0.5/dev-guide/source_zh_cn/error_handle/handle.html`.

[13] Paulo Emílio de Vilhena and François Pottier. A separation logic for effect handlers. *Proceedings of the ACM on Programming Languages*, 5(POPL), article 33, pages 1–28, 2021. https://doi.org/10.1145/3434314.

[14] Paulo Emílio de Vilhena and François Pottier. A type system for effect handlers and dynamic labels. In *Proceedings of ESOP*, volume 13990 of *LNCS*, pages 225–252. Springer, 2023. https://doi.org/10.1007/978-3-031-30044-8_9.

[15] Dan Frumin, Robbert Krebbers, and Lars Birkedal. ReLoC Reloaded: A mechanized relational logic for fine-grained concurrency and logical atomicity. *Logical Methods in Computer Science*, 17(3), article 9, pages 1–59, 2021. https://doi.org/10.46298/LMCS-17(3:9)2021.

[16] Glen Mével, Jacques-Henri Jourdan, and François Pottier. Cosmo: A concurrent separation logic for Multicore OCaml. *Proceedings of the ACM on Programming Languages*, 4(ICFP), article 96, pages 1–29, 2020. https://doi.org/10.1145/3408978.

[17] Jonas Kastberg Hinrichsen, Jesper Bengtson, and Robbert Krebbers. Actris 2.0: Asynchronous session-type based reasoning in separation logic. *Logical Methods in Computer Science*, 18(2), article 16, pages 1–64, 2022. https://doi.org/10.46298/LMCS-18(2:16)2022.

[18] Paulo Emílio de Vilhena, Simcha van Collem, Ines Wright, and Robbert Krebbers. A relational separation logic for effect handlers. *Proceedings of the ACM on Programming Languages*, 10(POPL), article 34, pages 981–1009, 2026. https://doi.org/10.1145/3776676. Rocq formalization: https://github.com/DeVilhena-Paulo/blaze.